Proceedings of Machine Learning Research vol 211:1–18, 2023 5th Annual Conference on Learning for Dynamics and Control

# Contrastive Example-Based Control

**Kyle Hatch** [1] KHATCH@STANFORD.EDU
**Benjamin Eysenbach** [2] BEYSENBA@CS.CMU.EDU
**Rafael Rafailov** [1] RAFAILOV@STANFORD.EDU
**Tianhe Yu** [1] TIANHEYU@CS.STANFORD.EDU
**Ruslan Salakhutdinov** [2] RSALAKHU@CS.CMU.EDU
**Sergey Levine** [3] SVLEVINE@EECS.BERKELEY.EDU
**Chelsea Finn** [1] CBFINN@CS.STANFORD.EDU
[1]*Department of Computer Science, Stanford University*
[2]*Machine Learning Department, Carnegie Mellon University*
[3]*Department of Electrical Engineering and Computer Sciences, UC Berkeley*

**Editors:** N. Matni, M. Morari, G. J. Pappas

arXiv:2307.13101v1 [cs.LG] 24 Jul 2023

## Abstract

While many real-world problems that might benefit from reinforcement learning, these problems rarely fit into the MDP mold: interacting with the environment is often expensive and specifying reward functions is challenging. Motivated by these challenges, prior work has developed data-driven approaches that learn entirely from samples from the transition dynamics and examples of high-return states. These methods typically learn a reward function from high-return states, use that reward function to label the transitions, and then apply an offline RL algorithm to these transitions. While these methods can achieve good results on many tasks, they can be complex, often requiring regularization and temporal difference updates. In this paper, we propose a method for offline, example-based control that learns an implicit model of multi-step transitions, rather than a reward function. We show that this implicit model can represent the Q-values for the example-based control problem. Across a range of state-based and image-based offline control tasks, our method outperforms baselines that use learned reward functions; additional experiments demonstrate improved robustness and scaling with dataset size.[1]

**Keywords:** reinforcement learning, offline RL, robot learning, reward learning, contrastive learning, model-based reinforcement learning, example-based control, reward-free learning

## 1. Introduction

Reinforcement learning is typically framed as the problem of maximizing a given reward function. However, in many real-world situations, it is more natural for users to define what they want an agent to do with examples of successful outcomes (Fu et al., 2018b; Zolna et al., 2020a; Xu and Denil, 2019; Eysenbach et al., 2021). For example, a user that wants their robot to pack laundry into a washing machine might provide multiple examples of states where the laundry has been packed correctly. This problem setting is often seen as a variant of inverse reinforcement learning (Fu et al., 2018b), where the aim is to learn only from examples of successful outcomes, rather than from

1. Videos of our method are available on the project website: https://sites.google.com/view/laeo-rl. Code is released at: https://github.com/khatch31/laeo.

© 2023 K. Hatch, B. Eysenbach, R. Rafailov, T. Yu, R. Salakhutdinov, S. Levine & C. Finn.

expert demonstrations. To solve this problem, the agent must both figure out what constitutes task success (i.e., what the examples have in common) and how to achieve such successful outcomes.

In this paper, our aim is to address this problem setting in the case where the agent must learn from offline data without trial and error. Instead, the agent must infer the outcomes of potential actions from the provided data, while also relating these inferred outcomes to the success examples. We will refer to this problem of offline RL with success examples as *offline example-based control*.

Most prior approaches involve two steps: *first* learning a reward function, and *second* combining it with an RL method to recover a policy (Fu et al., 2018b; Zolna et al., 2020a; Xu and Denil, 2019). While such approaches can achieve excellent results when provided sufficient data (Kalashnikov et al., 2021; Zolna et al., 2020a), learning the reward function is challenging when the number of success examples is small (Li et al., 2021; Zolna et al., 2020a). Moreover, these prior approaches are relatively complex (e.g., they use temporal difference learning) and have many hyperparameters.

Our aim is to provide a simple and scalable approach that avoids the challenges of reward learning. The main idea will be learning a certain type of dynamics model. Then, using that model to predict the probabilities of reaching each of the success examples, we will be able to estimate the Q-values for every state and action. Note that this approach does not use an offline RL algorithm as a subroutine. The key design decision is the model type; we will use an implicit model of the time-averaged future (precisely, the discounted state occupancy measure). This decision means that our model reasons across multiple time steps but will not output high-dimensional observations (only a scalar number). A limitation of this approach is that it will correspond to a single step of policy improvement: the dynamics model corresponds to the dynamics of the behavioral policy, not of the reward-maximizing policy. While this means that our method is not guaranteed to yield the optimal policy, our experiments nevertheless show that our approach outperforms multi-step RL methods.

The main contribution of this paper is an offline RL method (LAEO) that learns a policy from examples of high-reward states. The key idea behind LAEO is an implicit dynamics model, which represents the probability of reaching states at some point in the future. We use this model to estimate the probability of reaching examples of high-return states. LAEO is simpler yet more effective than prior approaches based on reward classifiers. Our experiments demonstrate that LAEO can successfully solve offline RL problems from examples of high-return states on four state-based and two image-based manipulation tasks. Our experiments show that LAEO is more robust to occlusions and also exhibits better scaling with dataset size than prior methods. We show that LAEO can work in example-based control settings in which goal-conditioned RL methods fail. Additionally, we show that the dynamics model learned by LAEO can generalize to multiple different tasks, being used to solve tasks that are not explicitly represented in the training data.

## 2. Related Work

**Reward learning.** To overcome the challenge of hand-engineering reward functions for RL, prior methods either use supervised learning or adversarial training to learn a policy that matches the expert behavior given by the demonstration (imitation learning) (Pomerleau, 1988; Ross et al., 2011; Ho and Ermon, 2016; Spencer et al., 2021) or learn a reward function from demonstrations and optimize the policy with the learned reward through trial and error (inverse RL) (Ng and Russell, 2000; Abbeel and Ng, 2004; Ratliff et al., 2006; Ziebart et al., 2008; Finn et al., 2016; Fu et al., 2018a). However, providing full demonstrations complete with agent actions is often difficult, therefore, recent works have focused on the setting where only a set of user-specified goal states or human videos

are available (Fu et al., 2018b; Singh et al., 2019; Kalashnikov et al., 2021; Xie et al., 2018; Eysenbach et al., 2021; Chen et al., 2021). These reward learning approaches have shown successes in real-world robotic manipulation tasks from high-dimensional imageinputs (Finn et al., 2016; Singh et al., 2019; Zhu et al., 2020; Chen et al., 2021). Nevertheless, to combat covariate shift that could lead the policy to drift away from the expert distribution, these methods usually require significant online interaction. Unlike these works that study online settings, we consider learning visuomotor skills from offline datasets.

**Offline RL.** Offline RL (Ernst et al., 2005; Riedmiller, 2005; Lange et al., 2012; Levine et al., 2020) studies the problem of learning a policy from a static dataset without online data collection in the environment, which has shown promising results in robotic manipulation (Kalashnikov et al., 2018; Mandlekar et al., 2020; Rafailov et al., 2021; Singh et al., 2020; Julian et al., 2020; Kalashnikov et al., 2021). Prior offline RL methods focus on the challenge of distribution shift between the offline training data and deployment using a variety of techniques, such as policy constraints (Fujimoto et al., 2018; Liu et al., 2020; Jaques et al., 2019; Wu et al., 2019; Zhou et al., 2020; Kumar et al., 2019; Siegel et al., 2020; Peng et al., 2019; Fujimoto and Gu, 2021; Ghasemipour et al., 2021), conservative Q-functions (Kumar et al., 2020; Kostrikov et al., 2021; Yu et al., 2021; Sinha and Garg, 2021), and penalizing out-of-distribution states generated by learned dynamics models (Kidambi et al., 2020; Yu et al., 2020b; Matsushima et al., 2020; Argenson and Dulac-Arnold, 2020; Swazinna et al., 2020; Rafailov et al., 2021; Lee et al., 2021; Yu et al., 2021).

While these prior works successfully address the issue of distribution shift, they still require reward annotations for the offline data. Practical approaches have used manual reward sketching to train a reward model (Cabi et al., 2019; Konyushkova et al., 2020; Rafailov et al., 2021) or heuristic reward functions (Yu et al., 2022). Others have considered offline learning from demonstrations, without access to a predefined reward function (Mandlekar et al., 2020; Zolna et al., 2020a; Xu et al., 2022; Jarboui and Perchet, 2021), however they rely on high-quality demonstration data. In contrast, our method: *(1)* addresses distributional shift induced by both the learned policy and the reward function in a principled way, *(2)* only requires user-provided goal states and *(3)* does not require expert-quality data, resulting in an effective and practical offline reward learning scheme.

## 3. Learning to Achieve Examples Offline

Offline RL methods typically require regularization, and our method will employ regularization in two ways. First, we regularize the policy with an additional behavioral cloning term, which penalizes the policy for sampling out-of-distribution actions. Second, our method uses the Q-function for the behavioral policy, so it performs one (not many) step of policy improvement. These regularizers mean that our approach is not guaranteed to yield the optimal policy.

### 3.1. Preliminaries

We assume that an agent interacts with an MDP with states $s \in \mathcal{S}$, actions $a$, a state-only reward function $r(s) \geq 0$, initial state distribution $p_0(s_0)$ and dynamics $p(s_{t+1} \mid s_t, a_t)$. We use $\tau = (s_0, a_0, s_1, a_1, \cdots)$ to denote an infinite-length trajectory. The likelihood of a trajectory under a policy $\pi(a \mid s)$ is $\pi(\tau) = p_0(s_0) \prod_{t=0}^{\infty} p(s_{t+1} \mid s_t, a_t)\pi(a_t \mid s_t)$. The objective is to learn a policy $\pi(a \mid s)$ that maximizes the expected, $\gamma$-discounted sum of rewards: $\max_\pi \mathbb{E}_{\pi(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t)\right]$. We define the Q-function for policy $\pi$ as the expected discounted sum of returns, conditioned on an

initial state and action:

$$Q^\pi(s,a) \triangleq \mathbb{E}_{\pi(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t) \middle| \begin{smallmatrix} s_0=s \\ a_0=a \end{smallmatrix}\right]. \tag{1}$$

We will focus on the offline (i.e., batch RL) setting. Instead of learning by interacting with the environment (i.e., via trial and error), the RL agent will receive as input a dataset of trajectories $\mathcal{D}_\tau = \{\tau \sim \beta(\tau)\}$ collected by a behavioral policy $\beta(a \mid s)$. We will use $Q^\beta(s,a)$ to denote the Q-function of the behavioral policy.

**Specifying the reward function.** In many real-world applications, specifying and measuring a scalar reward function is challenging, but providing examples of good states (i.e., those which would receive high rewards) is straightforward. Thus, we follow prior work (Fu et al., 2018b; Zolna et al., 2020a; Eysenbach et al., 2021; Xu and Denil, 2019; Zolna et al., 2020b) in assuming that the agent does not observe scalar rewards (i.e., $\mathcal{D}_\tau$ does not contain reward information). Instead, the agent receives as input a dataset $\mathcal{D}_* = \{s^*\}$ of high-reward states $s^* \in \mathcal{S}$. These high-reward states are examples of good outcomes, which the agent would like to achieve. The high-reward states are not labeled with their specific reward value.

To make the control problem well defined, we must relate these success examples to the reward function. We do this by assuming that the frequency of each success example is proportional to its reward: good states are more likely to appear (and be duplicated) as success examples.

**Assumption 1** *Let $p_\tau(s)$ be the empirical probability density of state $s$ in the trajectory dataset, and let $p_*(s)$ as the empirical probability density of state $s$ under the high-reward state dataset. We assume that there exists a positive constant $c$ such that $r(s) = c\frac{p_*(s)}{p_\tau(s)}$ for all states $s$.*

This is the same assumption as Eysenbach et al. (2021). This assumption is important because it shows how example-based control is universal: for any reward function, we can specify the corresponding example-based problem by constructing a dataset of success examples that are sampled according to their rewards. We assumed that rewards are non-negative so that these sampling probabilities are positive.

This assumption can also be read in reverse. When a user constructs a dataset of success examples in an arbitrary fashion, they are implicitly defining a reward function. In the tabular setting, the (implicit) reward function for state $s$ is the count of the times $s$ occurs in the dataset of success examples. Compared with goal-conditioned RL (Kaelbling, 1993), defining tasks via success examples is more general. By identifying what all the success examples have in common (e.g., laundry is folded), the RL agent can learn what is necessary to solve the task and what is irrelevant (e.g., the color of the clothes in the laundry). We now can define our problem statement as follows:

**Definition 1** *In the **offline example-based control** problem, a learning algorithm receives as input a dataset of trajectories $\mathcal{D}_\tau = \{\tau\}$ and a dataset of successful outcomes $\mathcal{D}_* = \{s\}$ satisfying Assumption 1. The aim is to output a policy that maximizes the RL objective (Eq. 3.1).*

This problem setting is appealing because it mirrors many practical RL applications: a user has access to historical data from past experience, but collecting new experience is prohibitively expensive. Moreover, this problem setting can mitigate the challenges of reward function design. Rather than having to implement a reward function and add instruments to measure the corresponding components, the users need only provide a handful of observations that solved the task. This problem

setting is similar to imitation learning, in the sense that the only inputs are data. However, unlike imitation learning, in this problem setting the high-reward states are not labeled with actions, and these high-reward states may not necessarily contain entire trajectories.

Our method will estimate the discounted state occupancy measure,

$$p^\beta(s_{t+} = s \mid s_0, a_0) \triangleq (1-\gamma) \sum_{t=0}^{\infty} \gamma^t p_t^\pi(s_t = s \mid s_0, a_0), \quad (2)$$

where $p_t^\beta(s_t \mid s, a)$ is the probability of policy $\beta(a \mid s)$ visiting state $s_t$ after exactly $t$ time steps. Unlike the transition function $p(s_{t+1} \mid s_t, a_t)$, the discounted state occupancy measure indicates the probability of visiting a state at any point in the future, not just at the immediate next time step. In tabular settings, this distribution corresponds to the successor representations (Dayan, 1993). To handle continuous settings, we will use the contrastive approach from recent work (Mazoure et al., 2020; Eysenbach et al., 2022). We will learn a function $f(s, a, s_f) \in \mathbb{R}$ takes as input an initial state-action pair as well as a candidate future state, and outputs a score estimating the likelihood that $s_f$ is a real future state. The loss function is a standard contrastive learning loss(e.g., Ma and Collins (2018)), where positive examples are triplets of a state, action, and future state:

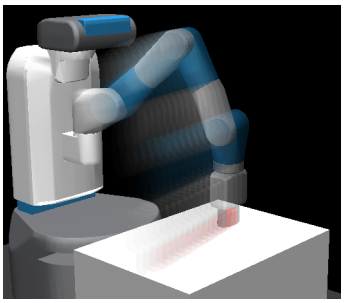

Figure 1: Our method will use contrastive learning to predict which states might occur at some point in the future.

$$\max_f \mathcal{L}(f; \mathcal{D}_\tau) \triangleq \mathbb{E}_{p(s,a), s_f \sim p^\beta(s_{t+}|s,a)} \left[\log \sigma(f(s,a,s_f))\right] + \mathbb{E}_{p(s,a), s_f \sim p(s)} \left[\log(1 - \sigma(f(s,a,s_f)))\right],$$

where $\sigma(\cdot)$ is the sigmoid function. At optimality, the implicit dynamics model encodes the discounted state occupancy measure:

$$f^*(s, a, s_f) = \log p^\beta(s_{t+} = s_f \mid s, a) - \log p_\tau(s_f). \quad (3)$$

We visualize this implicit dynamics model in Fig. 1. Note that this dynamics model is policy dependent. Because it is trained with data collected from one policy ($\beta(a \mid s)$), it will correspond to the probability that *that* policy visits states in the future. Because of this, our method will result in estimating the value function for the behavioral policy (akin to 1-step RL (Brandfonbrener et al., 2021)), and will not perform multiple steps of policy improvement. Intuitively, the training of this implicit model resembles hindsight relabeling (Kaelbling, 1993; Andrychowicz et al., 2017). However, it is generally unclear how to use hindsight relabeling for single-task problems. Despite being a single-task method, our method will be able to make use of hindsight relabeling to train the dynamics model.

### 3.2. Deriving Our Method

The key idea behind out method is that this implicit dynamics model can be used to represent the Q-values for the example-based problem, up to a constant. The proof is in Appendix A.

**Lemma 2** *Assume that the implicit dynamics model is learned without errors. Then the Q-function for the data collection policy $\beta(a \mid s)$ can be expressed in terms of this implicit dynamics model:*

$$Q^\beta(s, a) = \frac{c}{1-\gamma} \mathbb{E}_{p_*(s^*)} \left[ e^{f(s,a,s^*)} \right]. \quad (4)$$

So, after learning the implicit dynamics model, we can estimate the Q-values by averaging this model's predictions across the success examples. We will update the policy using Q-values estimated in this manner, plus a regularization term:

$$\min_{\pi} \mathcal{L}(\pi; f, \mathcal{D}_*) \triangleq -(1-\lambda)\mathbb{E}_{\pi(a|s)p(s),s^*\sim\mathcal{D}_*}\left[e^{f(s,a,s^*)}\right] - \lambda\mathbb{E}_{s,a\sim\mathcal{D}_\tau}\left[\log \pi(a \mid s)\right]. \quad (5)$$

In our experiments, we use a weak regularization coefficient of $\lambda = 0.5$.

It is worth comparing this approach to prior methods based on learned reward functions (Xu and Denil, 2019; Fu et al., 2018b; Zolna et al., 2020a). Those methods learn a reward function from the success examples, and use that learned reward function to synthetically label the dataset of trajectories. Both approaches can be interpreted as learning a function on one of the datasets and then applying that function to the other dataset. Because it is easier to fit a function when given large quantities of data, we predict that our approach will outperform the learned reward function approach when the number of success examples is small, relative to the number of unlabeled trajectories. Other prior methods (Eysenbach et al., 2021; Reddy et al., 2020) avoid learning reward functions by proposing TD update rules that are applied to both the unlabeled transitions and the high-return states. However, because these methods have yet to be adapted to the offline RL setting, we will focus our comparisons on the reward-learning methods.

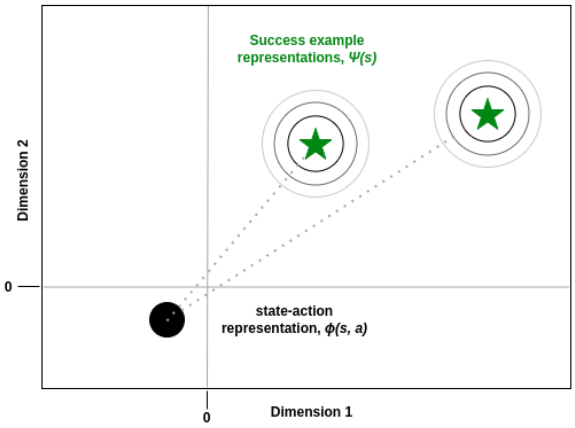


Figure 2: If the state-action representation $\phi(s, a)$ is close to the representation of a high-return state $\psi(s)$, then the policy is likely to visit that state. Our method estimates Q-values by combining the distances to all the high-return states (Eq. 1).

### 3.3. A Geometric Perspective

Before presenting the complete RL algorithm, we provide a geometric perspective on the representations learned by our method. Our implicit models learns a representation of state-action pairs $\phi(s, a)$ as well as a representation of future states $\psi(s)$. One way that our method can optimize these representations is by treating $\phi(s, a)$ as a prediction for the future representations.[2] Each of the high-return states can be mapped to the same representation space. To determine whether a state-action pair has a large or small Q-value, we can simply see whether the predicted representation $\phi(s, a)$ is close to the representations of any of the success examples. Our method learns these representations so that the Q-values are directly related to the Euclidean distances[3] from each success example. Thus, our method can be interpreted as learning a representation space such that estimating Q-values corresponds to simple geometric operations (kernel smoothing with an RBF kernel (Hastie et al., 2009, Chpt. 6)) on the learned representations. While the example-based control problem is more general than goal-conditioned RL (see Sec. 3.1), we can recover goal-conditioned RL as a special case by using a single success example.

2. Our method can also learn the opposite, where $\psi(s)$ is a prediction for the previous representations.
3. When representations are normalized, the dot product is equivalent to the Euclidean norm. We find that unnormalized features work better in our experiments.

### 3.4. A Complete Algorithm

We now build a complete offline RL algorithm based on these Q-functions. We will call our method LEARNING TO ACHIEVE EXAMPLES OFFLINE (LAEO). Our algorithm will resemble one-step RL methods, but differ in how the Q-function is trained. After learning the implicit dynamics model (and, hence, Q-function) we will optimize the policy. The objective for the policy is maximizing (log) Q-values plus a regularization term, which penalizes sampling unseen actions:[4]

$$\begin{aligned}\max_{\pi}\ &(1-\lambda)\log \mathbb{E}_{\pi(a|s)p_\tau(s)}\left[Q(s,a)\right] + \lambda \mathbb{E}_{(s,a)\sim p_\tau(s,a)}\left[\log \pi(a \mid s)\right] \\ &= (1-\lambda)\log \mathbb{E}_{\pi(a|s),s^*\sim p_*(s)}\left[e^{f(s,a,s^*)}\right] + \lambda \mathbb{E}_{(s,a)\sim p_\tau(s,a)}\left[\log \pi(a \mid s)\right]. \end{aligned} \tag{6}$$

As noted above, this is a one-step RL method: it updates the policy to maximize the Q-values of the behavioral policy. Performing just a single step of policy improvement can be viewed as a form of regularization in RL, in the same spirit as early stopping is a form of regularization in supervised learning. Prior work has found that one-step RL methods can perform well in the offline RL setting. Because our method performs only a single step of policy improvement, we are not guaranteed that it will converge to the reward-maximizing policy. We summarize the complete algorithm in Alg. 1.

**Algorithm 1** Learning to Achieve Examples Offline

1: **Inputs**: dataset of trajectories $\mathcal{D} = \{\tau\}$, dataset of high-return states $\mathcal{D}_* = \{s\}$.
2: Learn the model via contrastive learning: $f \leftarrow \arg\min_f \mathcal{L}(f; \mathcal{D}_\tau)$ ▷ Eq. 5
3: Learn the policy: $\pi \leftarrow \arg\min_\pi \mathcal{L}(\pi; f, \mathcal{D}_*)$ ▷ Eq. 6
4: **return** policy $\pi(a \mid s)$

## 4. Experiments

Our experiments test whether LAEO can effectively solve offline RL tasks that are specified by examples of high-return states, rather than via scalar reward functions. We study when our approach outperforms prior approaches based on learned reward functions. We look not only at the performance relative to baselines on state-based and image-based tasks, but also how that performance depends on the size and composition of the input datasets. Additional experiments study how LAEO performs when provided with varying numbers of success observations and whether our method can solve partially observed tasks. We include full hyperparameters and implementation details in Appendix B. Code is available at https://github.com/khatch31/laeo. Videos of our method are available at https://sites.google.com/view/laeo-rl.

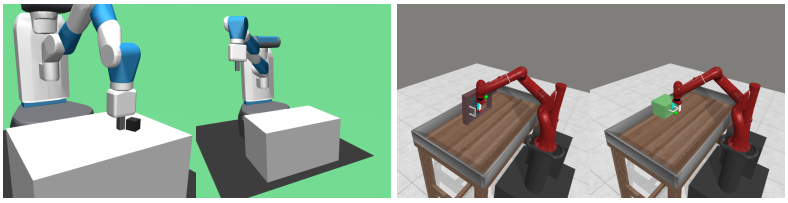

Figure 3: **Benchmark tasks**: We evaluate the performance of LAEO on six simulated manipulation tasks, two of which use pixel observations (`FetchReach-image` and `FetchPush-image`) and four of which use low-dimensional states (`FetchReach`, `FetchPush`, `SawyerWindowOpen`, and `SawyerDrawerClose` ).

4. For all experiments except Fig. 8, we apply Jensen's inequality to the first term, using $\mathbb{E}_{\pi(a|s),s^*\sim p_*(s)}[f(s,a,s^*)]$.

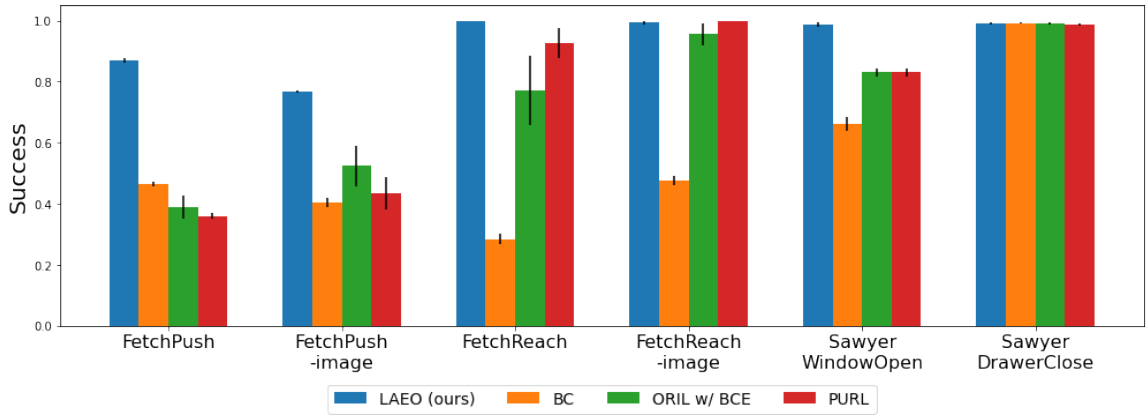


Figure 4: **Benchmark comparison**: LAEO matches or outperforms prior example-based offline RL methods on state and image-based tasks, including those that learn a separate reward function (ORIL, PURL).The gap in performance is most significant on the `FetchPush` and `FetchPush-image` tasks, which involve more complicated dynamics than the other tasks, suggesting that LAEO may outperform model free reward-learning approaches on tasks with complicated dynamics. LAEO also outperforms BC on all of the tasks, highlighting LAEO's ability to learn a policy that outperforms the behavior policy on non-demonstration datasets.

**Baselines.** Our main point of comparison will be prior methods that use learned reward functions: ORIL (Zolna et al., 2020a) and PURL (Xu and Denil, 2019). The main difference between these methods is the loss function used to train reward function: ORIL uses binary cross entropy loss while PURL uses a positive-unlabeled loss (Xu and Denil, 2019). Note that the ORIL paper also reports results using a positive-unlabeled loss, but for the sake of clarity we simply refer to it as PURL. After learning the reward function, each of these methods applies an off-the-shelf RL algorithm. We will implement all baselines using the TD3+BC (Fujimoto and Gu, 2021) offline RL algorithm. These offline RL methods achieve good performance on tasks specified via reward functions (Kostrikov et al., 2021; Brandfonbrener et al., 2021; Fujimoto and Gu, 2021). We also include Behavioral Cloning (BC) results.

**Benchmark comparison.** We start by comparing the performance of LAEO to these baselines on six manipulation tasks. `FetchReach` and `FetchPush` are two manipulation tasks from Plappert et al. (2018) that use state-based observations. `FetchReach-image` and `FetchPush-image` are the same tasks but with image-based observations. `SawyerWindowOpen` and `Sawyer-DrawerClose` are two manipulation tasks from Yu et al. (2020a). For each of these tasks, we collect a dataset of medium quality by training an online agent from Eysenbach et al. (2022) and rolling out multiple checkpoints during the course of training. The resulting datasets have success rates between $45\% - 50\%$. We report results after $500,000$ training gradient steps (or $250,000$ steps, if the task success rates have converged by that point).

We report results in Fig. 4. We observe that LAEO, PURL, and ORIL perform similarly on `FetchReach` and `FetchReach-image`. This is likely because these are relatively easy tasks, and each of these methods is able to achieve a high success rate. Note that all of these methods significantly outperform BC, indicating that they are able to learn better policies than the mode behavior policies represented in the datasets. On `SawyerDrawerClose`, all methods, including BC, achieve near perfect success rates, likely due to the simplicity of this task. On `FetchPush`, `FetchPush-image`, and `SawyerWindowOpen`, LAEO outperforms all of the baselines by a

significant margin. Recall that the main difference between LAEO and PURL/ORIL is by learning a dynamics model, rather than the reward function. These experiments suggest that for tasks with more complex dynamics, learning a dynamics model can achieve better performance than is achieved by model-free reward classifier methods.

**Varying the input data.** Our next experiment studies how the dataset composition affects LAEO and the baselines. On each of three tasks, we generate a low-quality dataset by rolling out multiple checkpoints from a partially trained agent from Eysenbach et al. (2022). In comparison to the medium-quality datasets collected earlier, which have success rates between $45\% - 50\%$, these low quality datasets have success rates between $8\% - 12\%$. We will denote these low quality datasets with the "Hard" suffix. Fig. 5 shows that LAEO continues to outperform baselines on these lower-quality datasets.

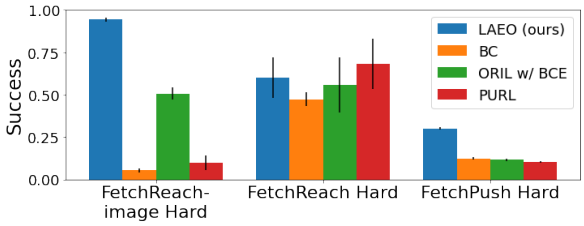


Figure 5: **Data quality.** LAEO continues to match or outperform reward classifier based methods on datasets that contain a low percentage of successful trajectories.

Our next experiments study how varying the number of high-return example states and the number of reward-free trajectories affects performance. As noted in the Sec. 1, we conjecture that our method will be especially beneficial relative to reward-learning approaches in settings with very few high-return example states. In Fig. 6 *(left)*, we vary the number of high-return example states on `FetchPush-image`, holding the number of unlabeled trajectories constant. We observe that LAEO maintains achieves the same performance with 1 success example as with 200 success examples. In contrast, ORIL's performance decreases as the number of high-return example states decreases. In Fig. 6 *(right)*, we vary the number of unlabeled trajectories, holding the number of high-return example states constant at 200. We test the performance of LAEO vs. ORIL on three different dataset sizes on `FetchPush-image`, roughly corresponding to three different orders of magnitude: the $0.1\times$ dataset contains $3,966$ trajectories, the $1\times$ dataset contains $31,271$ trajectories, and the $10\times$ dataset contains $300,578$ trajectories. We observe that LAEO continues to see performance gains as number of unlabeled trajectories increases, whereas ORIL's performance plateaus. Taken together these results suggest that, in comparison to reward classifier based methods, LAEO needs less human supervision and is more effective at leveraging large quantities of unlabeled data.

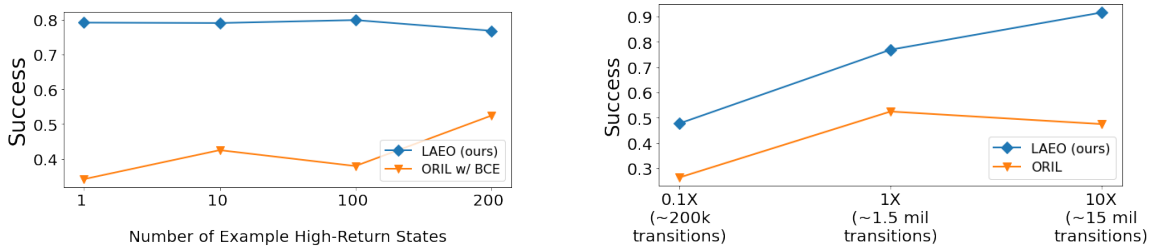


Figure 6: **Effect of dataset size**: *(Left)* The most competitive baseline (ORIL) achieves better performance when given more examples of high-return states, likely because it makes it easier to learn ORIL's reward classifier. LAEO, which does not require learning a reward classifier, consistently achieves high success rates. *(Right)* LAEO continues to improve when trained with more reward-free trajectories, while ORIL's performance plateaus.

**Partial Observability.** We also test the performance of LAEO on a partially-observed task. We modify the camera position in the `FetchPush-image` so that the block is occluded whenever the end effector is moved to touch the block. While such partial observability can stymie temporal difference methods (Whitehead and Ballard, 1991), we predict that LAEO might continue to solve this task because it does not rely on temporal difference learning. The results, shown in Fig. 7, confirm this prediction. On this partially observable task, we compare the performance of LAEO with that of ORIL, the best performing baseline on the fully observable tasks. On the partially observable task, LAEO achieves a success rate of 51.9%, versus 33.9% for ORIL.

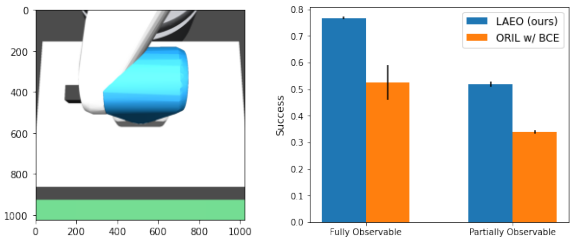


Figure 7: **Partial observability.** LAEO continues to solve the `FetchPush-image` manipulation task in a setting where the new camera placement causes partial observability. This camera angle causes the block to be hidden from view by the gripper when the gripper reaches down to push the block.

**Comparison to Goal-Conditioned RL.** One of the key advantages of example-based control, relative to goal-conditioned RL, is that the policy can identify common patterns in the success examples to solve tasks in scenarios where it has never before seen a success example. In settings such as robotics, this can be an issue since acquiring a goal state to provide to the agent requires already solving the desired task in the first place. We test this capability in a variant of the `SawyerDrawerClose` environment. For training, the drawer's X position is chosen as one of five fixed locations. Then, we evaluate the policy learned by LAEO on three types of environments: *In Distribution*: the drawer's X position is one of the five locations from training; *Interpolation*: The drawer's X position is between some of the locations seen during training; *Extrapolation*: The drawer's X position is outside the range of X positions seen during training. We compare to a goal-conditioned policy learned via contrastive RL, where actions are extracted by averaging over the (training) success examples: $\pi(a \mid s) = \mathbb{E}_{s^* \sim p_*(s)}[\pi(a \mid s, g = s^*)]$.

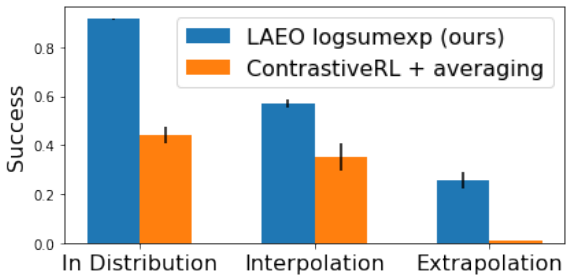


Figure 8: **Comparison with goal-conditioned RL.** LAEO solves manipulation tasks at multiple different locations without being provided with a goal-state at test time.

The results, shown in Fig. 8, show that LAEO consistently outperforms this goal-conditioned baseline. As expected, the performance is highest for the In Distribution environments and lowest for the Extrapolation environments. Taken together, these experiments show that LAEO can learn to reach multiple different goal locations without access to goal states during test time.

**Multitask Critic.** We explore whether a LAEO dynamics network trained on data from one task can be used to solve other downstream tasks. We create a simple multitask environment by defining several different tasks that can be solved in the `SawyerDrawerClose` environment: `Close`, `Half-closed`, `Open`, `Reach-near`, `Reach-medium`, and `Reach-far`. We then use a trained critic network from the previous set of experiments (Comparison to Goal-Conditioned RL), condition it on a success example from a downstream task, and select actions by using cross entropy method (CEM) optimization. By using CEM optimization, we do not need to train a sepa-

rate policy network for each of the tasks. See Appendix C for implementation details and for details of the multitask drawer environment.

CEM over the LAEO critic achieves non-zero success rates on all six tasks, despite only being trained on data from the `Close` task (see Figure 9). In contrast, randomly sampling actions from the action space achieves a 0% success rate on all of the tasks. Results are averaged across eight random seeds. This suggests that a single LAEO critic can be leveraged to solve multiple downstream tasks, as long as the dynamics required to solve those tasks are represented in the training data. Note that since we condition the critic network on a single goal example, these experiments can be interpreted from a goal-conditioned perspective as well as an example-based control perspective. In future work, we aim to explore the multitask capabilities of the LAEO dynamics model in an example-based control setting at a larger scale. This will involve training on larger, more diverse datasets as well as conditioning the critic network on multiple success examples for a single task (as done in the Comparison to Goal-Conditioned RL experiments).

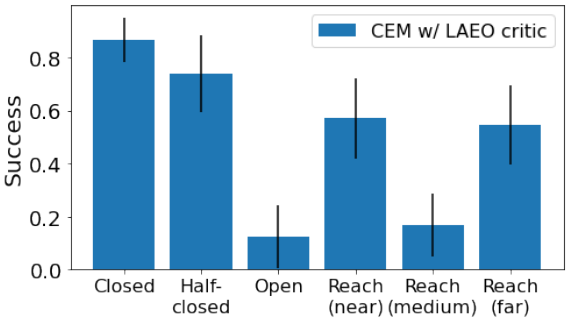


Figure 9: **Multitask Critic**: Cross entropy method (CEM) optimization over the LAEO dynamics model trained only on the data from the drawer close task is able to solve six different tasks. Randomly sampling actions from the action space results in a 0% success rate across all of the six tasks (not shown for clarity).

## 5. Conclusion

In this paper, we present an RL algorithm aimed at settings where data collection and reward specification are difficult. Our method learns from a combination of high-return states and reward-free trajectories, integrating these two types of information to learn reward-maximizing policies. Whereas prior methods perform this integration by learning a reward function and then applying an off-the-shelf RL algorithm, ours learns an implicit dynamics model. Not only is our method simpler (no additional RL algorithm required!), but also it achieves higher success rates than prior methods.

While our experiments only start to study the ability of contrastive-based methods to scale to high-dimensional observations, we conjecture that methods like LAEO may be particularly amenable to such problems because the method for learning the representations (contrastive learning) resembles prior representation learning methods (Mazoure et al., 2020; Nair et al., 2022). Scaling this method to very large offline datasets is an important direction for future work.

## 6. Acknowledgments

BE is supported by the Fannie and John Hertz Foundation and the NSF GRFP (DGE2140739).

## References

Pieter Abbeel and Andrew Y Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, page 1, 2004.

Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. *arXiv preprint arXiv:1707.01495*, 2017.

Arthur Argenson and Gabriel Dulac-Arnold. Model-based offline planning. *arXiv preprint arXiv:2008.05556*, 2020.

David Brandfonbrener, Will Whitney, Rajesh Ranganath, and Joan Bruna. Offline rl without off-policy evaluation. *Advances in Neural Information Processing Systems*, 34:4933–4946, 2021.

Serkan Cabi, Sergio Gómez Colmenarejo, Alexander Novikov, Ksenia Konyushkova, Scott Reed, Rae Jeong, Konrad Zołna, Yusuf Aytar, David Budden, Mel Vecerik, et al. A framework for data-driven robotics. *arXiv preprint arXiv:1909.12200*, 2019.

Annie S Chen, Suraj Nair, and Chelsea Finn. Learning generalizable robotic reward functions from" in-the-wild" human videos. *arXiv preprint arXiv:2103.16817*, 2021.

Peter Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural computation*, 5(4):613–624, 1993.

Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.

Benjamin Eysenbach, Sergey Levine, and Ruslan Salakhutdinov. Replacing rewards with examples: Example-based policy search via recursive classification, 2021.

Benjamin Eysenbach, Tianjun Zhang, Ruslan Salakhutdinov, and Sergey Levine. Contrastive learning as goal-conditioned reinforcement learning. *arXiv preprint arXiv:2206.07568*, 2022.

Chelsea Finn, Sergey Levine, and Pieter Abbeel. Guided cost learning: Deep inverse optimal control via policy optimization. In *International conference on machine learning*, pages 49–58. PMLR, 2016.

Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. *International Conference on Learning Representations*, 2018a.

Justin Fu, Avi Singh, Dibya Ghosh, Larry Yang, and Sergey Levine. Variational inverse control with events: A general framework for data-driven reward definition. *arXiv preprint arXiv:1805.11686*, 2018b.

Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. *arXiv preprint arXiv:2106.06860*, 2021.

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. *arXiv preprint arXiv:1812.02900*, 2018.

Seyed Kamyar Seyed Ghasemipour, Dale Schuurmans, and Shixiang Shane Gu. Emaq: Expected-max q-learning operator for simple yet effective offline and online rl. In *International Conference on Machine Learning*, pages 3682–3691. PMLR, 2021.

Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer, 2009.

Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. *Conference on Neural Information Processing Systems*, 2016.

Matt Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, et al. Acme: A research framework for distributed reinforcement learning. *arXiv preprint arXiv:2006.00979*, 2020.

Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, Craig Ferguson, Agata Lapedriza, Noah Jones, Shixiang Gu, and Rosalind Picard. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. *arXiv preprint arXiv:1907.00456*, 2019.

Firas Jarboui and Vianney Perchet. Offline inverse reinforcement learning, 2021. URL https://arxiv.org/abs/2106.05068.

Ryan Julian, Benjamin Swanson, Gaurav S Sukhatme, Sergey Levine, Chelsea Finn, and Karol Hausman. Efficient adaptation for end-to-end vision-based robotic manipulation. *arXiv preprint arXiv:2004.10190*, 2020.

Leslie Pack Kaelbling. Learning to achieve goals. In *IJCAI*, pages 1094–1099. Citeseer, 1993.

Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, et al. Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning*, pages 651–673. PMLR, 2018.

Dmitry Kalashnikov, Jacob Varley, Yevgen Chebotar, Benjamin Swanson, Rico Jonschkowski, Chelsea Finn, Sergey Levine, and Karol Hausman. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *arXiv preprint arXiv:2104.08212*, 2021.

Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. Morel: Model-based offline reinforcement learning. *arXiv preprint arXiv:2005.05951*, 2020.

Ksenia Konyushkova, Konrad Zolna, Yusuf Aytar, Alexander Novikov, Scott Reed, Serkan Cabi, and Nando de Freitas. Semi-supervised reward learning for offline reinforcement learning. *Offline Reinforcement Learning Workshop at Neural Information Processing Systems*, 2020.

Ilya Kostrikov, Jonathan Tompson, Rob Fergus, and Ofir Nachum. Offline reinforcement learning with fisher divergence critic regularization. *arXiv preprint arXiv:2103.08050*, 2021.

Aviral Kumar, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. Stabilizing off-policy q-learning via bootstrapping error reduction. In *Advances in Neural Information Processing Systems*, pages 11761–11771, 2019.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *arXiv preprint arXiv:2006.04779*, 2020.

Sascha Lange, Thomas Gabel, and Martin A. Riedmiller. Batch reinforcement learning. In *Reinforcement Learning*, volume 12. Springer, 2012.

Byung-Jun Lee, Jongmin Lee, and Kee-Eung Kim. Representation balancing offline model-based reinforcement learning. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=QpNz8r_Ri2Y.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

Kevin Li, Abhishek Gupta, Ashwin Reddy, Vitchyr H Pong, Aurick Zhou, Justin Yu, and Sergey Levine. Mural: Meta-learning uncertainty-aware rewards for outcome-driven reinforcement learning. In *International Conference on Machine Learning*, pages 6346–6356. PMLR, 2021.

Yao Liu, Adith Swaminathan, Alekh Agarwal, and Emma Brunskill. Provably good batch reinforcement learning without great exploration. *arXiv preprint arXiv:2007.08202*, 2020.

Zhuang Ma and Michael Collins. Noise contrastive estimation and negative sampling for conditional models: Consistency and statistical efficiency. *arXiv preprint arXiv:1809.01812*, 2018.

Ajay Mandlekar, Fabio Ramos, Byron Boots, Silvio Savarese, Li Fei-Fei, Animesh Garg, and Dieter Fox. Iris: Implicit reinforcement without interaction at scale for learning control from offline robot manipulation data. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4414–4420. IEEE, 2020.

Tatsuya Matsushima, Hiroki Furuta, Yutaka Matsuo, Ofir Nachum, and Shixiang Gu. Deployment-efficient reinforcement learning via model-based offline optimization. *arXiv preprint arXiv:2006.03647*, 2020.

Bogdan Mazoure, Remi Tachet des Combes, Thang Long Doan, Philip Bachman, and R Devon Hjelm. Deep reinforcement and infomax learning. *Advances in Neural Information Processing Systems*, 33:3686–3698, 2020.

Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*, 2022.

Andrew Y. Ng and Stuart J. Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, ICML '00, 2000.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.

Matthias Plappert, Marcin Andrychowicz, Alex Ray, Bob McGrew, Bowen Baker, Glenn Powell, Jonas Schneider, Josh Tobin, Maciek Chociej, Peter Welinder, et al. Multi-goal reinforcement learning: Challenging robotics environments and request for research. *arXiv preprint arXiv:1802.09464*, 2018.

Dean A Pomerleau. Alvinn: an autonomous land vehicle in a neural network. In *Proceedings of the 1st International Conference on Neural Information Processing Systems*, pages 305–313, 1988.

Rafael Rafailov, Tianhe Yu, A. Rajeswaran, and Chelsea Finn. Offline reinforcement learning from images with latent space models. *Learning for Decision Making and Control (L4DC)*, 2021.

Nathan D. Ratliff, J. Andrew Bagnell, and Martin A. Zinkevich. Maximum margin planning. In *Proceedings of the 23rd International Conference on Machine Learning*, ICML '06, 2006.

Siddharth Reddy, Anca D. Dragan, and Sergey Levine. Sqil: Imitation learning via reinforcement learning with sparse rewards. *International Conference on Learning Representations*, 2020.

Martin Riedmiller. Neural fitted q iteration–first experiences with a data efficient neural reinforcement learning method. In *European Conference on Machine Learning*, pages 317–328. Springer, 2005.

Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *AISTATS*, 2011.

Noah Y Siegel, Jost Tobias Springenberg, Felix Berkenkamp, Abbas Abdolmaleki, Michael Neunert, Thomas Lampe, Roland Hafner, and Martin Riedmiller. Keep doing what worked: Behavioral modelling priors for offline reinforcement learning. *arXiv preprint arXiv:2002.08396*, 2020.

Avi Singh, Larry Yang, Kristian Hartikainen, Chelsea Finn, and Sergey Levine. End-to-end robotic reinforcement learning without reward engineering. *arXiv preprint arXiv:1904.07854*, 2019.

Avi Singh, Albert Yu, Jonathan Yang, Jesse Zhang, Aviral Kumar, and Sergey Levine. Cog: Connecting new skills to past experience with offline reinforcement learning. *arXiv preprint arXiv:2010.14500*, 2020.

Samarth Sinha and Animesh Garg. S4rl: Surprisingly simple self-supervision for offline reinforcement learning. *arXiv preprint arXiv:2103.06326*, 2021.

Jonathan Spencer, Sanjiban Choudhury, Arun Venkatraman, Brian Ziebart, and J. Andrew Bagnell. Feedback in imitation learning: The three regimes of covariate shift. *ArXiv Preprint*, 2021.

Phillip Swazinna, Steffen Udluft, and Thomas Runkler. Overcoming model bias for robust offline deep reinforcement learning. *arXiv preprint arXiv:2008.05533*, 2020.

Steven D Whitehead and Dana H Ballard. Learning to perceive and act by trial and error. *Machine Learning*, 7(1):45–83, 1991.

Yifan Wu, George Tucker, and Ofir Nachum. Behavior regularized offline reinforcement learning. *arXiv preprint arXiv:1911.11361*, 2019.

Annie Xie, Avi Singh, Sergey Levine, and Chelsea Finn. Few-shot goal inference for visuomotor learning and planning. In *Conference on Robot Learning*, pages 40–52. PMLR, 2018.

Danfei Xu and Misha Denil. Positive-unlabeled reward learning. *arXiv preprint arXiv:1911.00459*, 2019.

Haoran Xu, Xianyuan Zhan, Honglei Yin, and Huiling Qin. Discriminator-weighted offline imitation learning from suboptimal demonstrations. *International Conference on Machine Learning*, 2022.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning*, pages 1094–1100. PMLR, 2020a.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. *arXiv preprint arXiv:2005.13239*, 2020b.

Tianhe Yu, Aviral Kumar, Rafael Rafailov, Aravind Rajeswaran, Sergey Levine, and Chelsea Finn. Combo: Conservative offline model-based policy optimization. *arXiv preprint arXiv:2102.08363*, 2021.

Tianhe Yu, Aviral Kumar, Yevgen Chebotar, Karol Hausman, Chelsea Finn, and Sergey Levine. How to leverage unlabeled data in offline reinforcement learning. *International Conference on Machine Learning*, 2022.

Wenxuan Zhou, Sujay Bajracharya, and David Held. Plas: Latent action space for offline reinforcement learning. *arXiv preprint arXiv:2011.07213*, 2020.

Henry Zhu, Justin Yu, Abhishek Gupta, Dhruv Shah, Kristian Hartikainen, Avi Singh, Vikash Kumar, and Sergey Levine. The ingredients of real-world robotic reinforcement learning. *arXiv preprint arXiv:2004.12570*, 2020.

Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.

Konrad Zolna, Alexander Novikov, Ksenia Konyushkova, Caglar Gulcehre, Ziyu Wang, Yusuf Aytar, Misha Denil, Nando de Freitas, and Scott Reed. Offline learning from demonstrations and unlabeled experience. *arXiv preprint arXiv:2011.13885*, 2020a.

Konrad Zolna, Scott Reed, Alexander Novikov, Sergio Gomez Colmenarejo, David Budden, Serkan Cabi, Misha Denil, Nando de Freitas, and Ziyu Wang. Task-relevant adversarial imitation learning. *Conference on Robot Learning*, 2020b.

## Appendix A. Proofs

The proof follows by substituting Assumption 1 into the definition of Q-values (Eq. 1):
**Proof**

$$
\begin{aligned}
Q^\beta(s,a) = \mathbb{E}_{\beta(\tau)}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t)\middle|_{a_0=a}^{s_0=s}\right] &= \frac{1}{1-\gamma}\int p^\beta(s_{t+}=s^* \mid s,a) r(s^*) ds^* \\
&= \frac{1}{1-\gamma}\int p^\beta(s_{t+}=s^* \mid s,a) c\frac{p_*(s^*)}{p_\tau(s^*)} ds^* \\
&= \frac{c}{1-\gamma}\int p_*(s^*) e^{f(s,a,s^*)} ds^* = \frac{c}{1-\gamma}\mathbb{E}_{s^*\sim p_*(s)}\left[e^{f(s,a,s^*)}\right].
\end{aligned}
$$

■

## Appendix B. Experimental Details

We implemented our method and all baselines using the ACME framework (Hoffman et al., 2020).

- Batch size: 1024 for state based experiments, 256 for image based experiments
- Training iterations: $250,000$ if task success rates had converged by that point, otherwise $500,000$
- Representation dimension: 256
- Reward learning loss (for baselines): binary cross entropy (for ORIL) and positive unlabeled (for PURL)
- Critic architecture: Two-layer MLP with hidden sizes of 1024. ReLU activations used between layers.
- Reward function architecture (for baselines): Two-layer MLP with hidden sizes of 1024. ReLU activations used between layers.
- Actor learning rate: $3 \times 10^{-4}$
- Critic learning rate: $3 \times 10^{-4}$
- Reward learning rate (for baselines): $1 \times 10^{-4}$
- $\lambda$ for behavioral cloning weight in policy loss term: $0.5$
- $\eta$ for PU loss: $0.5$
- Size of offline datasets: Each dataset on `Fetch` tasks the consists of approximately $4,000$ trajectories of length 50, except for the `FetchPush-image` dataset, which consists of approximately $40,000$ trajectories. Each dataset on the `Sawyer` tasks consists of approximately $4,000$ trajectories of length 200.

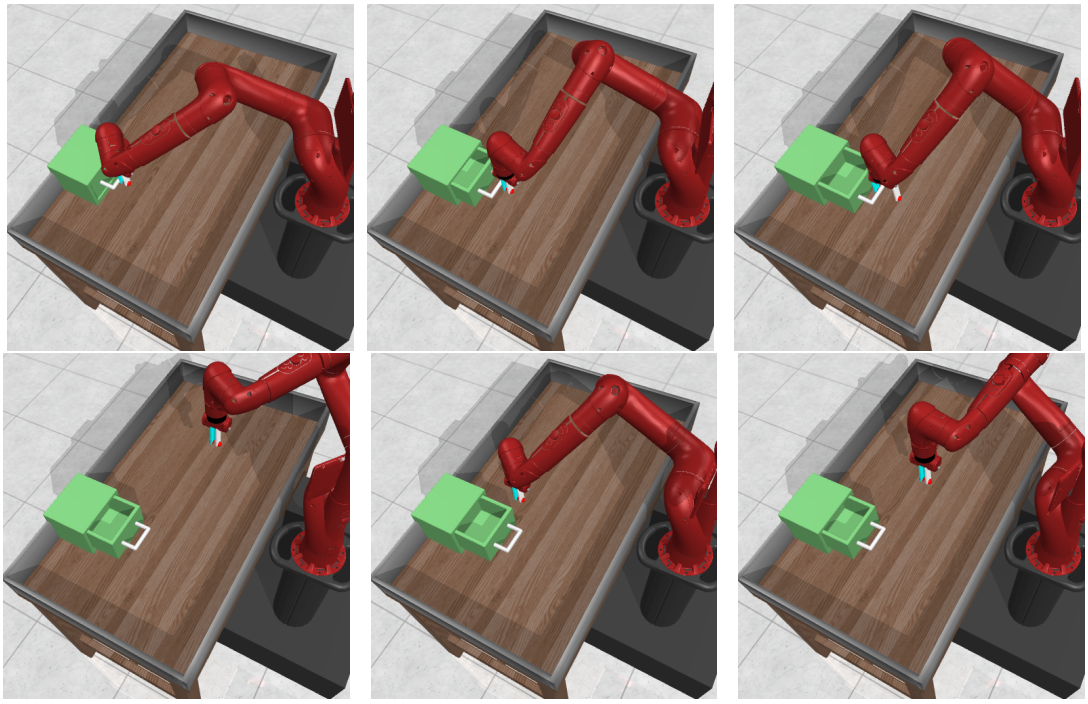

Figure 10: **Multitask Drawer Environment.**: We apply cross entropy method (CEM) optimization on the LAEO dynamics model trained only on the data from the drawer close task to solve six different tasks: close, half-closed, open, reach (near), reach (medium), and reach (far). .

## Appendix C. Multitask Critic Experiments

The `Half-closed` task requires the agent to push the drawer from an open position into a halfway closed position. The `Open` task requires the agent to pull the drawer from a closed position into an open position. The `Close` task is the same as in the original `SawyerDrawerClose` environment, and requires the agent to push the drawer from an opened position into a closed position. The three reaching tasks, `Reach-near`, `Reach-medium`, `Reach-far`, require the agent to reach the end-effector to a three different target positions. The tasks are visualized in Figure 10.

For these experiments, we load the final checkpoint of a critic network from the previous set of experiments (Comparison to Goal-Conditioned RL), and select actions by using cross entropy method (CEM) optimization on the critic network. By using CEM optimization, we do not need to train a separate policy network for each of the tasks. Since the LAEO dynamics model is a multistep dynamics model (meaning that the model predicts whether a goal state will be reached sometime in the future and not just at the subsequent timestep) we are able to use CEM directly with the dynamics model. Specifically, for each task, we collect a success example using a scripted policy from Yu et al. (2020a). Then, at each environment timestep $t$, we condition the LAEO dynamics model on the success example, and then run CEM to choose an action that maximizes the output of the dynamics network. At each timestep, we perform 10 iterations of CEM, using a population size of $10,000$ and an elite population size of $2,000$. Results are averaged across eight random seeds.